

LoRA: Low-Rank Adaptation of Large Language Models

Background

Motivation

- Full fine-tuning is expensive and scales poorly
- Question: Can LoRA achieve comparable performance on GLUE?

LoRA (Hu et al., 2022)

- Freeze backbone model, inject **low-rank matrices** (A, B)

Project Goal

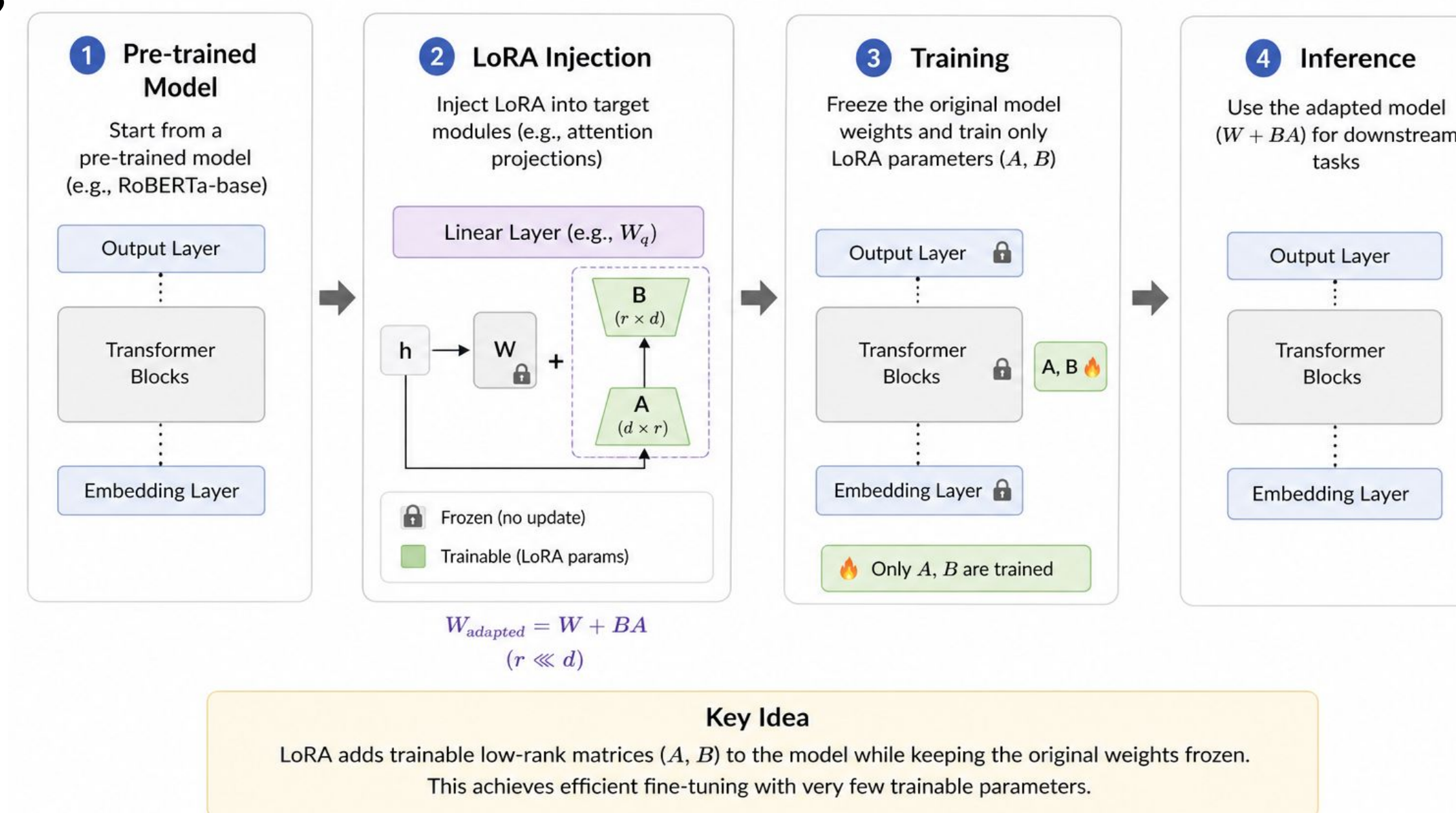
- Re-implement LoRA on RoBERTa-base (Liu et al., 2019)
- LoRA vs Full fine-tuning vs Frozen backbone
- The effect of LoRA rank, alpha and target modules

Project Setup 🤖

- Model: RoBERTa-base
- Dataset: GLUE benchmark tasks (MNLI, MRPC, QNLI, QQP, SST-2, CoLA)
- Target Modules: Query, Value
- Compute Resource: NVIDIA A800, PyTorch+HuggingFace
- Batch size=32, Training epochs=3, Max Sequence Length=128

Methodology

LoRA Pipeline (Simplified)



Future Work

- Expand the efficiency analysis by measuring GPU memory usage and storage cost in addition to trainable parameters and runtime.
- Extend the experiments to larger backbone models to test whether the same trends hold beyond RoBERTa-base.
- Run more multi-seed experiments to improve result stability, especially on smaller and more variable tasks.



Experiment Results

Method	MNLI	MRPC	QNLI	QQP	SST-2	CoLA
Frozen Backbone	53.34	68.38	66.30	75.14	80.62	69.13
Full Fine-tuning	85.98	87.25	91.09	80.41	91.97	82.64
LoRA	84.95	70.59	90.48	87.36	93.46	79.39

Table 1: Performance comparison of Frozen Backbone, Full Fine-tuning, and LoRA on GLUE tasks.

Why was our initial MRPC result much lower than the LoRA paper?

1 LoRA Paper	2 Our initial run	3 Our rerun
MRPC score: 89.7 Reference result	Accuracy: 70.6% F1: 81.8% Avg(Acc, F1): 76.2 3 epochs LR not tuned	Accuracy: 87.7% F1: 91.0% Avg(Acc, F1): 89.4 30 epochs tuned LR
Model: RoBERTa-base (LoRA) Metric: Avg(Accuracy, F1)	Model: RoBERTa-base (LoRA) Trainable parameters: 887,042	Model: RoBERTa-base (LoRA) Trainable parameters: 887,042



Takeaway

The gap mainly comes from training setup, not LoRA itself.
Using more epochs and a tuned learning rate brought our result much closer to the paper.

Extension

Effect of LoRA Hyperparameters on MNLI Accuracy

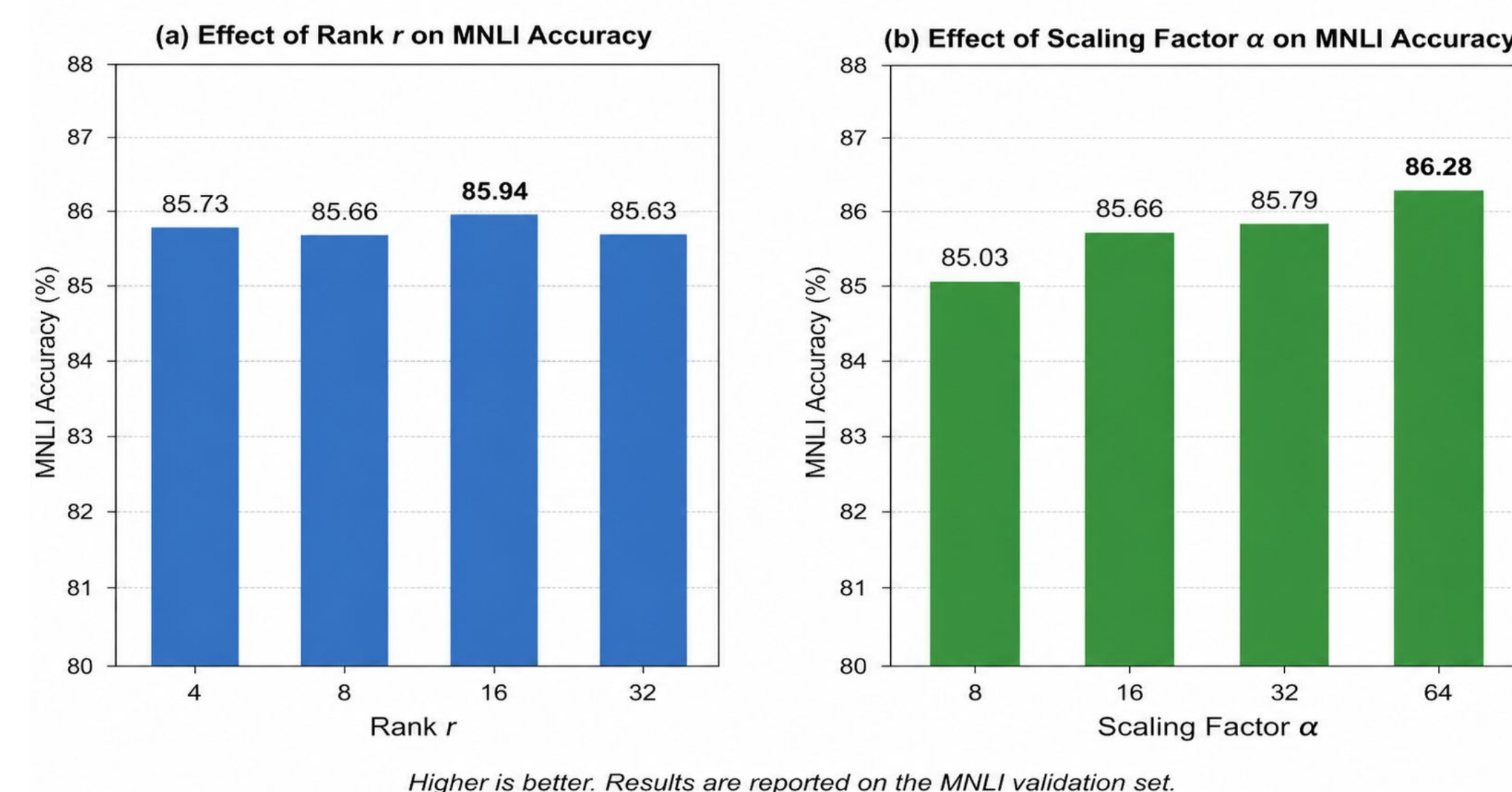


Table 7: MNLI performance comparison for different LoRA target modules.

LoRA Target Modules	Eval Loss	Accuracy	Precision	Recall	F1
query	0.4694	81.43%	81.31%	81.32%	81.31%
query, value	0.3932	84.92%	84.80%	84.83%	84.81%
query, value, key, dense	0.3716	85.92%	85.82%	85.84%	85.83%

Conclusion

Overall Performance on GLUE benchmark

- LoRA achieves performance close to, and sometimes better than full fine-tuning on multiple tasks (QQP, SST-2).

Performance Efficiency

- LoRA uses only about 0.89M trainable parameters (0.71%), a roughly 140x reduction vs full fine-tuning.

Effect of hyperparameters

- Accuracy improves steadily as alpha increases from 8 to 64
- Increasing rank does not guarantee monotonic improvement
- Best weight modules in practice: query+value.

References

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR.
- Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. (2018). GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.